

Event Management Service Coordination System

1. Introduction

The Event Management Service Coordination System is designed to support an event manager in organizing and coordinating multiple service teams for events such as weddings, corporate gatherings, and private parties. The system enables the event manager to create event plans, assign tasks to various teams, track preparation progress, and send automated reminders and alerts to ensure smooth execution leading up to the event day.

The solution provides a structured, automated workflow for communication between the event manager and service teams, reducing manual coordination effort, improving event timelines, and ensuring all teams stay aligned with expectations.

2. Problem Statement

Event coordination often involves numerous teams and strict timelines. Manual communication can lead to delays, overlooked tasks, poor coordination, and last-minute issues. To address these challenges, the event manager needs a system that can:

- Create and manage event details (type, date, expected guests).
- Assign tasks to service teams with clear instructions and deadlines.
- Send automated notifications and preparation reminders.
- Request periodic task status updates.
- Immediately alert the event manager if issues or delays arise.
- Send final event-day alerts to ensure all teams are prepared.

This system aims to simplify team communication, improve tracking, and enable proactive issue handling.

3. Architecture Overview

The system follows a high-level microservice-based architecture that supports modularity, scalability, and asynchronous communication. The major components include:

Frontend / Application Layer

Interface for the event manager to create events, assign tasks, track progress, and handle issues.

API Layer (Play Framework)

Provides REST endpoints for event creation, task assignment, and progress tracking.

Service Layer

Handles business logic:

- Event creation
- Task assignment workflows
- Timeline-based notifications and reminders
- Issue escalation logic

Database Layer (MySQL)

Stores persistent data, including:

- Event details
- Task assignments
- Team information
- Task statuses
- Notifications log

Notification Service (Akka Actors)

Manages automated reminders and real-time alerts using:

- Scheduled reminders
- Progress update triggers

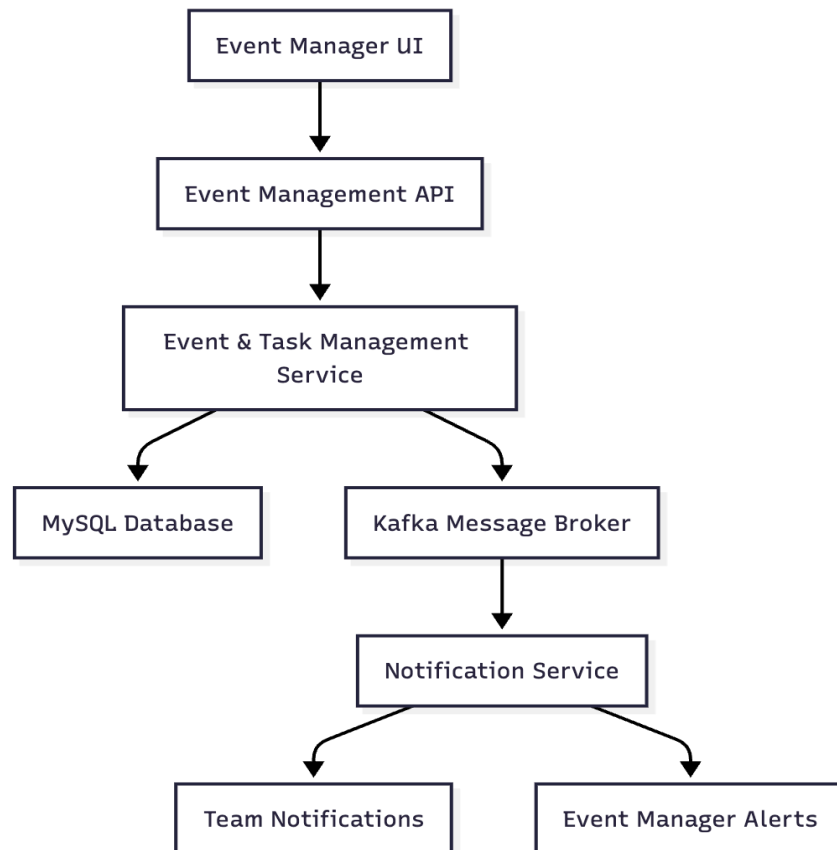
- Issue alerts
- Event-day notifications

Message Broker (Kafka)

Used for reliable asynchronous communication between the main application and Notification Service.

Deployment Layer (Docker)

Offers consistent, portable, and scalable deployment across all environments.



4. Component Design

Frontend (Event Manager Interface)

- Create event plans.
- Assign tasks to teams.
- Provide instructions & deadlines.
- View progress status.
- Receive issue alerts.

Play API Controllers

- Accept REST API requests from frontend.
- Validate requests.
- Forward data to Service Layer.

Service Layer

- Apply business rules for event creation and task assignment.
- Publish task-related events to Kafka (task assigned, issue raised, reminder needed).
- Coordinate with Notification Service for reminders and alerts.

Database (MySQL)

Includes the following structured tables:

- **Events:** Event type, date, guest count.
- **Tasks:** Specific team assignments.
- **Teams:** Catering, Decorations, Entertainment, Logistics.
- **Reminders:** Scheduled events and logs.
- **Issue Reports:** From teams to event manager.

Notification Service (Akka)

- Uses actors for concurrent notification processing.
- Schedulers trigger:
 - Task assignment notifications
 - Preparation reminders (1 day before + few hours before)
 - Progress check-ins

- Event-day alerts
- Detects issue events from Kafka and notifies event manager immediately.

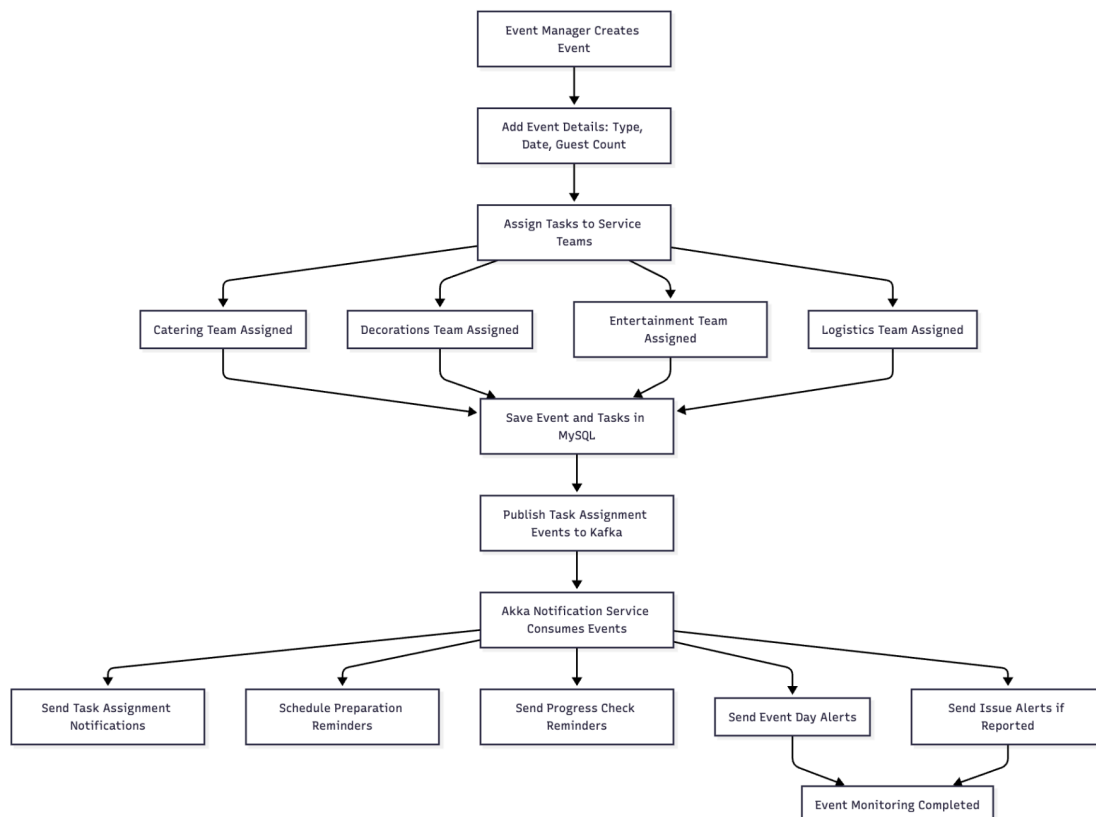
Kafka Messaging Layer

- Ensures reliable, asynchronous delivery of notification events.
- Guarantees no loss of reminders or alerts.

Docker Deployment

- Each service packaged as a separate container.
- Allows independent scaling (e.g., Notification Service under heavy load).

5. System Workflow



6. Microservices Design

1. Event Management Service

Purpose:

Manages all event operations:

- Event creation
- Task assignment
- Progress update handling
- Issue reporting

Responsibilities:

- Hosting all REST APIs
- Coordinating event workflows
- Publishing task-related events to Kafka

2. Notification Service

Purpose:

Handles all automated reminders and alerts.

Responsibilities:

- Consume Kafka events
- Send task assignment notifications
- Send preparation reminders
- Send progress check-in reminders
- Issue escalation alerts
- Final event-day alerts

Inter-Service Communication

- **Synchronous:**
Frontend → Event Management Service (REST)
- **Asynchronous:**
Event Management Service → Notification Service (Kafka)

7.Data Base Design

```
CREATE TABLE event_users (id BIGINT AUTO_INCREMENT PRIMARY KEY, username VARCHAR(50) UNIQUE, password VARCHAR(255), user_role VARCHAR(20), email VARCHAR(100), created_at TIMESTAMP, updated_at TIMESTAMP);
```

```
CREATE TABLE teams (id BIGINT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50) UNIQUE, created_at TIMESTAMP, updated_at TIMESTAMP);
```

```
CREATE TABLE events (id BIGINT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100), type VARCHAR(20), date TIMESTAMP, guest_count INT, created_by BIGINT, created_at TIMESTAMP, updated_at TIMESTAMP, UNIQUE KEY unique_event (name, date), FOREIGN KEY (created_by) REFERENCES event_users(id));
```

```
CREATE TABLE team_users (id BIGINT AUTO_INCREMENT PRIMARY KEY, team_id BIGINT, user_id BIGINT, FOREIGN KEY (team_id) REFERENCES teams(id), FOREIGN KEY (user_id) REFERENCES event_users(id));
```

```
CREATE TABLE tasks (id BIGINT AUTO_INCREMENT PRIMARY KEY, event_id BIGINT, team_id BIGINT, description TEXT, status VARCHAR(20), start_time TIMESTAMP, end_time TIMESTAMP, special_request TEXT, created_at TIMESTAMP, updated_at TIMESTAMP, UNIQUE KEY unique_task (event_id, team_id, description(255)), FOREIGN KEY (event_id) REFERENCES events(id) ON DELETE CASCADE, FOREIGN KEY (team_id) REFERENCES teams(id));
```

```
INSERT INTO event_users (id, username, email, password, user_role, created_at, updated_at) VALUES (1,'shekar','naresh996463@gmail.com','Shekar@9705','EVENT_MANAGER','2025-11-20 17:11:48','2025-11-20 17:11:48'),(6,'jane.doe','naresh119935@gmail.com','SecurePass123!','TEAM_MEMBER','2025-11-21 11:54:11','2025-11-21 11:54:11'),(7,'rohith','nareshnagula61@gmail.com','Rohith@123!','TEAM_MEMBER','2025-11-21 23:54:20','2025-11-21 23:54:20');
```

```
INSERT INTO teams (id, name, created_at, updated_at) VALUES (5,'Catering Team','2025-11-21 18:39:21','2025-11-21 18:39:21'),(6,'Decorations Team','2025-11-21 23:43:06','2025-11-21 23:43:06');
```

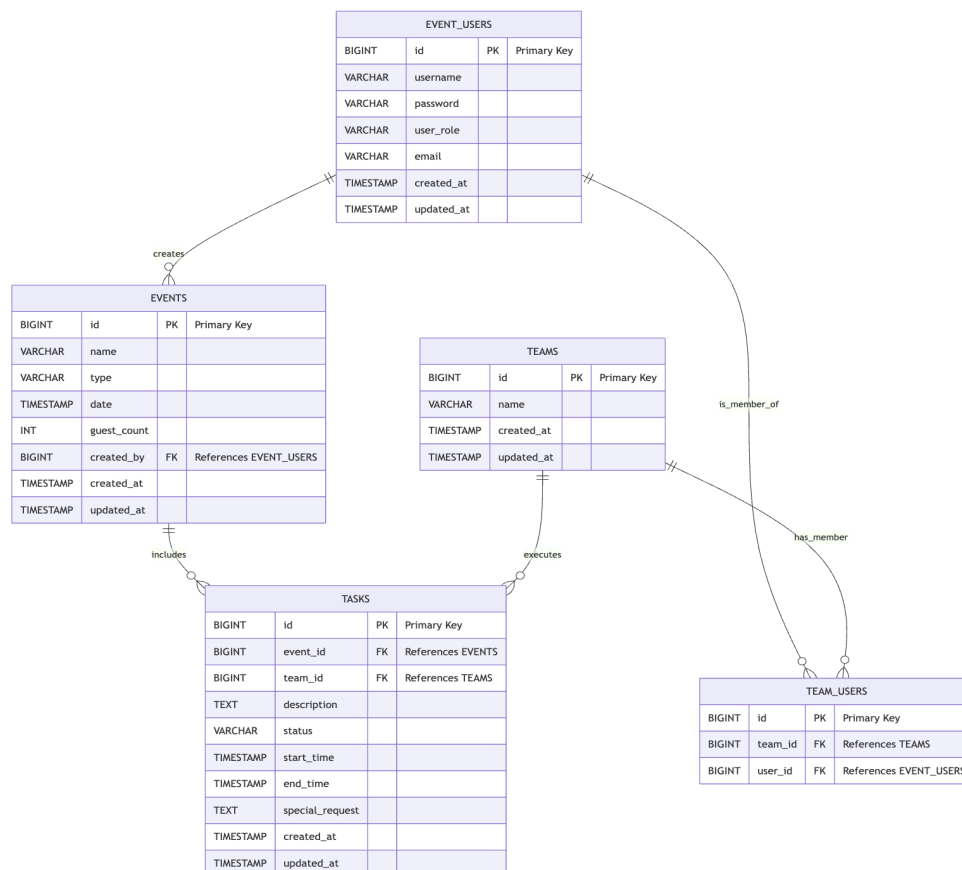
```
INSERT INTO events (id, name, type, date, guest_count, created_by, created_at, updated_at) VALUES (5,'John and Jane Wedding','Wedding','2025-12-10 10:00:00',100,6,'2025-11-21 18:41:35','2025-11-21 18:41:35'),(8,'Sweety','BirthDay','2025-12-10 10:00:00',100,6,'2025-11-21 23:51:05','2025-11-21 23:51:05');
```

```
INSERT INTO team_users (id, team_id, user_id) VALUES (6,5,6),(8,5,7),(7,6,7);
```

```
INSERT INTO tasks (id, event_id, team_id, description, status, start_time, end_time, special_request, created_at, updated_at) VALUES (1,5,5,'Prepare appetizers and main course','Pending','2025-12-10 08:00:00','2025-12-10 12:00:00','Include vegan options','2025-11-22 10:00:00','2025-11-22 10:00:00'),(2,5,6,'Decorate hall and stage','Pending','2025-12-10 07:00:00','2025-12-10 11:00:00','Use red and white theme','2025-11-22 10:05:00','2025-11-22 10:05:00'),(3,8,5,'Prepare birthday cake and snacks','Pending','2025-12-10 08:30:00','2025-12-10 12:30:00','Chocolate cake with candles','2025-11-22 10:10:00','2025-11-22 10:10:00'),(4,8,6,'Set up balloons and
```

decorations','Pending','2025-12-10 07:30:00','2025-12-10 11:30:00','Blue and yellow theme','2025-11-22 10:15:00','2025-11-22 10:15:00'),(5,5,5,'Arrange drinks and beverages','Pending','2025-12-10 09:00:00','2025-12-10 12:00:00','Include soft drinks','2025-11-22 10:20:00','2025-11-22 10:20:00'),(6,5,6,'Set up photo booth and props','Pending','2025-12-10 09:00:00','2025-12-10 13:00:00','Include backdrop with couple's name','2025-11-22 10:25:00','2025-11-22 10:25:00'),(7,8,5,'Prepare party favors','Pending','2025-12-10 10:00:00','2025-12-10 13:00:00','Small gift bags for kids','2025-11-22 10:30:00','2025-11-22 10:30:00'),(8,8,6,'Decorate birthday cake table','Pending','2025-12-10 08:30:00','2025-12-10 11:00:00','Add balloons around table','2025-11-22 10:35:00','2025-11-22 10:35:00');

INSERT INTO notifications (id, task_id, type, sent_at, status, created_at, updated_at) VALUES (1,1,'Email',NULL,'Pending','2025-11-22 10:00:00','2025-11-22 10:00:00'),(2,2,'Email',NULL,'Pending','2025-11-22 10:05:00','2025-11-22 10:05:00'),(3,3,'Email',NULL,'Pending','2025-11-22 10:10:00','2025-11-22 10:10:00'),(4,4,'Email',NULL,'Pending','2025-11-22 10:15:00','2025-11-22 10:15:00'),(5,5,'Email',NULL,'Pending','2025-11-22 10:20:00','2025-11-22 10:20:00'),(6,6,'Email',NULL,'Pending','2025-11-22 10:25:00','2025-11-22 10:25:00'),(7,7,'Email',NULL,'Pending','2025-11-22 10:30:00','2025-11-22 10:30:00'),(8,8,'Email',NULL,'Pending','2025-11-22 10:35:00','2025-11-22 10:35:00');



8. REST API Endpoints

Event Management Service API

Authentication

1. Login

URL: POST — <http://localhost:9000/api/login>

Input:

```
{  
  "username": "Shekar",  
  "password": "Shekar@9705"  
}
```

Output:

```
{  
  "status": "success",  
  "message": "Login successful",  
  "data": {  
    "token":  
    "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyYb2x1IjoiYWRTaW4iLCJleHAiOiE3NjM4MjY0MzMsInVzZXIiOiJhZG1pbiIsIm1hdCI6MTc2MzgyMjgzM30.YwsJ5pb0BVWPtsra7XdgFoju8RJ8iMrih8_zzPsbIWA",  
    "expires_in": 3600  
  }  
}
```

Case fail:

POST http://localhost:9000/api/login

Docs Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "username": "admin",
3   "password": "Admin@12322"
4 }
```

Body Cookies Headers (4) Test Results (0/1) 401 Unauthorized 209 ms 1

{ } JSON Preview Debug with AI

```
1 {
2   "status": "fail",
3   "message": "Invalid username or password"
4 }
```

Case Success:

POST http://localhost:9000/api/login

Docs Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "username": "admin",
3   "password": "Admin@123"
4 }
```

Body Cookies Headers (4) Test Results (1/1) 200 OK 312 ms 378 B

{ } JSON Preview Visualize

```
1 {
2   "status": "success",
3   "message": "Login successful",
4   "data": {
5     "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyY2x1IjoieWRtaW4iLCJleHAiOjE3NjM4MjIzOTgsInVzZXIiOiJhZG1pb2IiLCJhdCI6MTc2Mzg4ODc5OH0.MQ8U7VejT-vMKck9SZPXsvQncHicWaImVwU8lig0TAQ",
6     "expires_in": 3600
7   }
8 }
```

User Management

Create User

URL: POST <http://localhost:9000/api/event-users/create>

Input:

```
{  
  
  "username": "rakesh",  
  
  "email": "rakesh21@gmail.com",  
  
  "password": "Rakesh@123!",  
  
  "userRole": "TEAM_MEMBER"  
}
```

OUTPUTS:

```
{  
  
  "status": "fail",  
  
  "message": "Username or email already exists"  
}  
  
{  
  
  "status": "success",  
  
  "message": "User created successfully",  
  
  "data": {  
  
    "id": 8,  
  
    "username": "rakesh",  
  
    "email": "rakesh21@gmail.com",  
  
    "password": "Rakesh@123!",  
  
    "userRole": "TEAM_MEMBER",  
  
    "createdAt": "2025-11-23 00:53:46.707",  
  
    "updatedAt": "2025-11-23 00:53:46.707"  
  }  
}
```

POST http://localhost:9000/api/event-users/create

Docs Params Authorization Headers (9) Body Scripts Settings

Headers 8 hidden

	Key	Value	Description
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyYb...	
	Key	Value	Description

Body Cookies Headers (4) Test Results 200 OK 4

{ } JSON Preview Visualize

```
1 {
2   "status": "success",
3   "message": "User created successfully",
4   "data": {
5     "id": 8,
6     "username": "rakesh",
7     "email": "rakesh21@gmail.com",
8     "password": "Rakesh@123!",
9     "userRole": "TEAM_MEMBER",
10    "createdAt": "2025-11-23 00:53:46.707",
11    "updatedAt": "2025-11-23 00:53:46.707"
12  }
13 }
```

POST http://localhost:9000/api/event-users/create

Docs Params Authorization Headers (9) Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "username": "rakesh",
3   "email": "rakesh21@gmail.com",
4   "password": "Rakesh@123!",
5   "userRole": "TEAM_MEMBER"
6 }
```

Body Cookies Headers (4) Test Results 200 OK 2

{ } JSON Preview Visualize

```
1 {
2   "status": "fail",
3   "message": "Username or email already exists"
4 }
```

Get All Users

URL: GET http://localhost:9000/api/event-users

GET

http://localhost:9000/api/event-users

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Headers

6 hidden

	Key	Value	Description	...	Bull
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyYb...			
	Key	Value	Description		

Body

Cookies

Headers (4)

Test Results

↺

200 OK • 189 ms •

{ } JSON

Preview

Visualize

↺

```
1 {
2   "status": "success",
3   "message": "Users fetched successfully",
4   "data": [
5     {
6       "id": 1,
7       "username": "shekar",
8       "email": "naresh996463@gmail.com",
9       "password": "Shekar@9705",
10      "userRole": "EVENT_MANAGER",
11      "createdAt": "2025-11-20 17:11:48.0",
12      "updatedAt": "2025-11-20 17:11:48.0"
13    },
14    {
15      "id": 6,
16      "username": "jane.doe",
17      "email": "naresh119935@gmail.com",
18      "password": "SecurePass123!",
19      "userRole": "TEAM_MEMBER",
20      "createdAt": "2025-11-21 11:54:11.0",
21      "updatedAt": "2025-11-21 11:54:11.0"
22    },
23  ]
24 }
```

Get User by ID

URL: GET <http://localhost:9000/api/event-users/1>

GET <http://localhost:9000/api/event-users/1>

Docs Params Authorization **Headers (7)** Body Scripts Settings

Headers 6 hidden

	Key	Value	Description
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyYb...	
	Key	Value	Description

Body Cookies Headers (4) Test Results 200 OK

{ } JSON Preview Visualize

```
1 {
2   "status": "success",
3   "message": "User fetched successfully",
4   "data": {
5     "id": 1,
6     "username": "shekar",
7     "email": "naresh996463@gmail.com",
8     "password": "Shekar@9705",
9     "userRole": "EVENT_MANAGER",
10    "createdAt": "2025-11-20 17:11:48.0",
11    "updatedAt": "2025-11-20 17:11:48.0"
12  }
13 }
```

Update User

URL: PUT <http://localhost:9000/api/event-users/7>

PUT <http://localhost:9000/api/event-users/8>

Docs Params Authorization Headers (9) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "email": "rakesh56@gmail.com"
3 }
```

Body Cookies Headers (4) Test Results 200 OK

{ } JSON Preview Visualize

```
1 {
2   "status": "success",
3   "message": "User updated successfully"
4 }
```

Delete User

URL: DELETE <http://localhost:9000/api/event-users/8>

DELETE ▼ <http://localhost:9000/api/event-users/8>

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Headers 6 hidden

	Key	Value	Description
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyYb...	
	Key	Value	Description

Body

Cookies

Headers (4)

Test Results

↺

200 OK

{}

JSON ▼

▶ Preview 🖼️ Visualize ▼

```
1 {
2   "status": "success",
3   "message": "User deleted successfully"
4 }
```

Event Management

Get All Events

URL: GET http://localhost:9000/api/events

GET

http://localhost:9000/api/events

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Headers

6 hidden

	Key	Value	Description
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyYb...	
	Key	Value	Description

Body

Cookies

Headers (4)

Test Results

🔄

200 OK • 200ms

{}

JSON

Preview

Visualize

⌵

```
1 {
2   "status": "success",
3   "message": "Events fetched successfully",
4   "data": [
5     {
6       "id": 5,
7       "name": "John and Jane Wedding",
8       "eventType": "Wedding",
9       "date": "2025-12-10 10:00:00.0",
10      "guestCount": 100,
11      "createdBy": 6,
12      "createdAt": "2025-11-21 18:41:35.0",
13      "updatedAt": "2025-11-21 18:41:35.0"
14    },
15    {
16      "id": 8,
17      "name": "Sweety",
18      "eventType": "BirthDay",
19      "date": "2025-12-10 10:00:00.0",
20      "guestCount": 100,
21      "createdBy": 6,
22      "updatedAt": "2025-11-21 18:41:35.0"
```

Event Management System | v1.0

Get Event By ID

URL: GET http://localhost:9000/api/events/5

GET

▼

http://localhost:9000/api/events/5

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Headers

6 hidden

	Key	Value	Description
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyYb...	
	Key	Value	Description

Body

Cookies

Headers (4)

Test Results

↺

200 OK

{ } JSON ▼

▶ Preview

📄 Visualize ▼

```
1  {
2    "status": "success",
3    "message": "Event fetched successfully",
4    "data": {
5      "id": 5,
6      "name": "John and Jane Wedding",
7      "eventType": "Wedding",
8      "date": "2025-12-10 10:00:00.0",
9      "guestCount": 100,
10     "createdBy": 6,
11     "createdAt": "2025-11-21 18:41:35.0",
12     "updatedAt": "2025-11-21 18:41:35.0"
13   }
14 }
```

Create an Event

URL: POST <http://localhost:9000/api/events/create>

Input:

```
{
  "name": "Mani",
  "eventType": "BirthDay",
  "date": "2025-12-10 10:00:00",
  "guestCount": 120
}
```

Output:

```
{
  "status": "success",
  "message": "Event created successfully",
  "data": {
    "id": 20,
    "name": "Mani",
    "eventType": "BirthDay",
    "date": "2025-12-10 10:00:00.0",
    "guestCount": 120,
    "createdBy": 6,
    "createdAt": "2025-11-23 01:45:11.078",
    "updatedAt": "2025-11-23 01:45:11.078"
  }
}
```

POST

http://localhost:9000/api/events/create

Docs

Params

Authorization

Headers (9)

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

```
1 {
2   "name": "Anual Party",
3   "eventType": "corporate event",
4   "date": "2025-12-10 10:00:00",
5   "guestCount": 100
6 }
7
```

Body

Cookies

Headers (4)

Test Results

200 OK

{ } JSON

Preview

Visualize

```
1 {
2   "status": "success",
3   "message": "Event created successfully",
4   "data": {
5     "id": 18,
6     "name": "Anual Party",
7     "eventType": "corporate event",
8     "date": "2025-12-10 10:00:00.0",
9     "guestCount": 100,
10    "createdBy": 1,
11    "createdAt": "2025-11-23 01:34:39.352",
12    "updatedAt": "2025-11-23 01:34:39.352"
13  }
14 }
```

Update an Event

URL: PUT <http://localhost:9000/api/events/18>

PUT http://localhost:9000/api/events/18

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "eventType": "corporate event",
3   "date": "2025-12-10 10:00:00.0",
4   "guestCount": 150
5 }
```

Body Cookies Headers (4) Test Results 200 OK

{ JSON Preview Visualize

```
1 {
2   "status": "success",
3   "message": "Event updated successfully"
4 }
```

Delete an Event

URL: DELETE http://localhost:9000/api/events/20

DELETE http://localhost:9000/api/events/20

Docs Params Authorization Headers (7) Body Scripts Settings

Headers 6 hidden

	Key	Value	Description
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyYb...	
	Key	Value	Description

Body Cookies Headers (4) Test Results 200 OK

{ JSON Preview Visualize

```
1 {
2   "status": "success",
3   "message": "Event deleted successfully"
4 }
```

Team Management

Get All Teams

URL: GET http://localhost:9000/api/teams

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:9000/api/teams
- Headers (7):** Authorization (checked), Key, Value, Description.
- Body:** JSON format, showing a successful response with status "success", message "Teams fetched successfully", and a list of two teams.
- Status:** 200 OK

The JSON response body is as follows:

```
1 {
2   "status": "success",
3   "message": "Teams fetched successfully",
4   "data": [
5     {
6       "id": 5,
7       "name": "Catering Team",
8       "createdAt": "2025-11-21 18:39:21.0",
9       "updatedAt": "2025-11-21 18:39:21.0"
10    },
11    {
12      "id": 6,
13      "name": "Decorations Team",
14      "createdAt": "2025-11-21 23:43:06.0",
15      "updatedAt": "2025-11-21 23:43:06.0"
16    }
17  ]
18 }
```

Get Team By ID

URL: GET http://localhost:9000/api/teams/5

GET

http://localhost:9000/api/teams/5

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Headers

6 hidden

	Key	Value	Description
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyb...	
	Key	Value	Description

Body

Cookies

Headers (4)

Test Results

🕒

200 OK

{ } JSON

Preview

Visualize

```
1  {
2    "status": "success",
3    "message": "Team fetched",
4    "data": {
5      "id": 5,
6      "name": "Catering Team",
7      "createdAt": "2025-11-21 18:39:21.0",
8      "updatedAt": "2025-11-21 18:39:21.0"
9    }
10 }
```

Create a Team

URL: POST <http://localhost:9000/api/events/create>

Input:

```
{  
  "name": "Entertainment Team"  
}
```

Output:

```
{  
  "status": "success",  
  "message": "Team created successfully",  
  "data": {  
    "id": 7,  
    "name": "Entertainment Team",  
    "createdAt": "2025-11-23 01:51:55.49",  
    "updatedAt": "2025-11-23 01:51:55.49"  
  }  
}
```

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** `http://localhost:9000/api/teams/create`
- Body Type:** raw (selected)
- Request Body (JSON):**

```
{  
  "name": "Entertainment Team"  
}
```
- Response Status:** 200 OK
- Response Body (JSON):**

```
{  
  "status": "success",  
  "message": "Team created successfully",  
  "data": {  
    "id": 7,  
    "name": "Entertainment Team",  
    "createdAt": "2025-11-23 01:51:55.49",  
    "updatedAt": "2025-11-23 01:51:55.49"  
  }  
}
```

Update a Team

URL: PUT <http://localhost:9000/api/teams/5>

Input:

```
{  
  "name": "Logistics Team"  
}
```

Output:

```
{  
  "status": "success",  
  "message": "Team updated successfully"  
}
```

The screenshot shows a REST client interface. At the top, the method is set to 'PUT' and the URL is 'http://localhost:9000/api/teams/5'. Below this, there are tabs for 'Docs', 'Params', 'Authorization', 'Headers (9)', 'Body', 'Scripts', and 'Settings'. The 'Body' tab is selected, and the content type is set to 'JSON'. The request body is a JSON object:

```
{  
  "name": "Logistics Team"  
}
```

. At the bottom, the response is shown with a status of '200 OK'. The response body is also a JSON object:

```
{  
  "status": "success",  
  "message": "Team updated successfully"  
}
```

Delete a Team

URL: DELETE <http://localhost:9000/api/teams/8>

The screenshot shows a REST client interface. At the top, the method is set to 'DELETE' and the URL is 'http://localhost:9000/api/teams/8'. Below this, there are tabs for 'Docs', 'Params', 'Authorization', 'Headers (7)', 'Body', 'Scripts', and 'Settings'. The 'Body' tab is selected, and the content type is set to 'JSON'. The request body is empty. At the bottom, the response is shown with a status of '200 OK'. The response body is a JSON object:

```
{  
  "status": "success",  
  "message": "Team deleted successfully"  
}
```


Team Assignment

Team assignment

URL: POST <http://localhost:9000/api/teams/assign-users>

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** <http://localhost:9000/api/teams/assign-users>
- Headers:** 9 headers are listed, with 8 hidden. The visible header is:

Key	Value	Description
Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyYb...	
Key	Value	Description
- Body:** JSON format. The response is:

```
1 {
2   "status": "success",
3   "message": "2 user(s) successfully assigned to team 9"
4 }
```
- Status:** 200 OK

Team Unassign

URL: POST <http://localhost:9000/api/teams/assign/remove>

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** <http://localhost:9000/api/teams/assign/remove>
- Body:** raw format. The request body is:

```
1 {
2   "teamId": 9,
3   "userIds": [9]
4 }
```
- Status:** 200 OK

The response body (JSON) is also visible below the request body:

```
1 {
2   "status": "success",
3   "message": "1 user(s) removed from team 9"
4 }
```

Task Management

Get All Tasks

URL: GET http://localhost:9000/api/tasks

GET

http://localhost:9000/api/tasks

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Headers

6 hidden

	Key	Value	Description
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJy...	
	Key	Value	Description

Body

Cookies

Headers (4)

Test Results

↺

200 OK

{}

JSON

Preview

Visualize

↕

```
1 {
2   "status": "success",
3   "message": "Tasks fetched successfully",
4   "data": [
5     {
6       "id": 1,
7       "eventId": 5,
8       "teamId": 5,
9       "description": "Prepare appetizers for guests",
10      "status": "Pending",
11      "startTime": "2025-12-10 08:00:00.0",
12      "endTime": "2025-12-10 09:30:00.0",
13      "specialRequest": "Vegetarian options included",
14      "createdAt": "2025-11-21 20:47:31.0",
15      "updatedAt": "2025-11-21 20:47:31.0"
16    },
17    {
18      "id": 4,
19      "eventId": 8,
20      "teamId": 6,
21      "description": "Decorate the hall with flowers and lights",
22      "status": "Pending",
23      "startTime": "2025-11-22 07:48:25.0",
24      "endTime": "2025-11-24 23:00:00.0"
```

Get Task By ID

URL: GET <http://localhost:9000/api/tasks/1>

GET

http://localhost:9000/api/tasks/1

Docs

Params

Authorization

Headers (7)

Body

Scripts

Settings

Headers

6 hidden

	Key	Value	Description
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyYb...	
	Key	Value	Description

Body

Cookies

Headers (4)

Test Results

200 OK

{}

JSON

Preview

Visualize

```
1  {
2    "status": "success",
3    "message": "Task fetched",
4    "data": {
5      "id": 1,
6      "eventId": 5,
7      "teamId": 5,
8      "description": "Prepare appetizers for guests",
9      "status": "Pending",
10     "startTime": "2025-12-10 08:00:00.0",
11     "endTime": "2025-12-10 09:30:00.0",
12     "specialRequest": "Vegetarian options included",
13     "createdAt": "2025-11-21 20:47:31.0",
14     "updatedAt": "2025-11-21 20:47:31.0"
15   }
16 }
```

Create a Task

URL: POST <http://localhost:9000/api/tasks/create>

Input:

```
{
  "eventId": 18,
  "teamId": 9,
  "description": "Prepare and serve catering for the Annual Party",
  "status": "Pending",
  "startTime": "2025-11-20 22:05:00",
  "endTime": "2025-11-20 23:00:00",
}
```

```
      "specialRequest": "Include vegetarian and non-vegetarian options, setup
buffet table",
      "createdAt": "2025-11-20 22:00:00",
      "updatedAt": "2025-11-20 22:00:00"
    }
  }
}
```

Output:

```
{
  "status": "success",
  "message": "Task created successfully",
  "data": {
    "id": 8,
    "eventId": 18,
    "teamId": 9,
    "description": "Prepare and serve catering for the Annual Party",
    "status": "Pending",
    "startTime": "2025-11-20 22:05:00.0",
    "endTime": "2025-11-20 23:00:00.0",
    "specialRequest": "Include vegetarian and non-vegetarian options,
setup buffet table",
    "reminderSent": false,
    "eventDayAlertSent": false,
    "createdAt": "2025-11-23 03:23:26.799",
    "updatedAt": "2025-11-23 03:23:26.799"
  }
}
```

POST <http://localhost:9000/api/tasks/create>

Docs Params Authorization Headers (9) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```

1  {
2    "eventId": 18,
3    "teamId": 9,
4    "description": "Prepare and serve catering for the Annual Party",
5    "status": "Pending",
6    "startTime": "2025-11-20 22:05:00",
7    "endTime": "2025-11-20 23:00:00",
8    "specialRequest": "Include vegetarian and non-vegetarian options, setup buffet table",
9    "createdAt": "2025-11-20 22:00:00",
10   "updatedAt": "2025-11-20 22:00:00"
11 }
12

```

Body Cookies Headers (4) Test Results | ↻ **200 OK** • 8

{} JSON ▾ ▷ Preview 🖼 Visualize ▾

```

1  {
2    "status": "success",
3    "message": "Task created successfully",
4    "data": {
5      "id": 5,
6      "eventId": 18,
7      "teamId": 9,
8      "description": "Prepare and serve catering for the Annual Party",
9      "status": "Pending",
10     "startTime": "2025-11-20 22:05:00.0",
11     "endTime": "2025-11-20 23:00:00.0",
12     "specialRequest": "Include vegetarian and non-vegetarian options, setup buffet table",
13     "reminderSent": false,
14     "eventDayAlertSent": false,
15     "createdAt": "2025-11-23 03:14:10.838",
16     "updatedAt": "2025-11-23 03:14:10.838"
17   }
18 }

```

POST <http://localhost:9000/api/tasks/create>

Docs Params Authorization Headers (9) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```

1  {
2    "eventId": 18,
3    "teamId": 9,
4    "description": "Prepare and serve catering for the Annual Party",
5    "status": "Pending",
6    "startTime": "2025-12-09 12:00:00",
7    "endTime": "2025-12-09 18:00:00",
8    "specialRequest": "Include vegetarian and non-vegetarian options, setup buffet table",
9    "createdAt": "2025-11-23 10:15:00",
10   "updatedAt": "2025-11-23 10:15:00"
11 }

```

Body Cookies Headers (4) Test Results | ↻ **200 OK** •

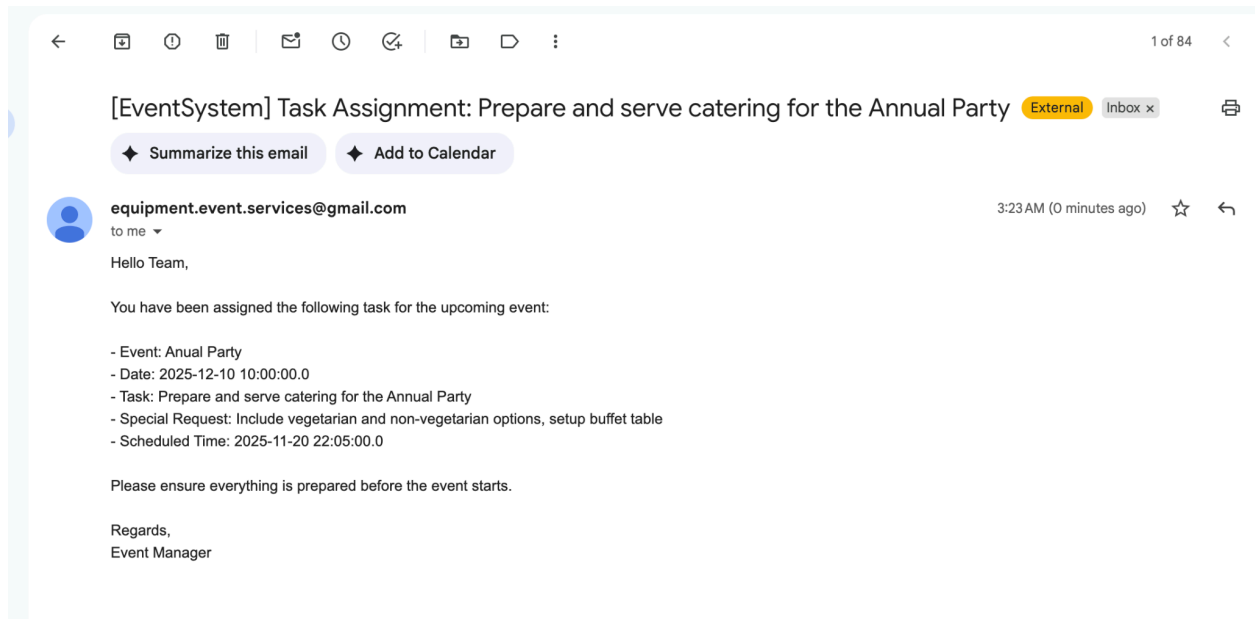
{} JSON ▾ ▷ Preview 🖼 Visualize ▾

```

1  {
2    "status": "fail",
3    "message": "Task already exists"
4  }

```

After task allocation, the task is created and notifications are sent to all assigned team members.



Update a Task

URL: PUT <http://localhost:9000/api/tasks/10>

Input:

```
{
  "status": "InProgress",
  "startTime": "2025-12-09 09:00:00"
}
```

Output:

```
{
  "status": "success",
  "message": "Task updated successfully"
}
```



Task Status Update: Decorate event hall

Inbox



equipment... 4:04 AM

to me ▾



Hello Team,

Task Update for your assigned task:

Event: Anual Party

Date: 2025-12-10 10:00:00.0

Task: Decorate event hall

Please take necessary action if required.

Regards,
Event Manager

Delete Task

URL: DELETE http://localhost:9000/api/tasks/11

DELETE ⌵ http://localhost:9000/api/tasks/11

Docs Params Authorization Headers (7) Body Scripts Settings

Headers 6 hidden

	Key	Value	Description
☒	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyb...	
	Key	Value	Description

Body Cookies Headers (4) Test Results ↺ 200 OK

{ } JSON ⌵ ▶ Preview 🖼 Visualize ⌵

```
1 {  
2   "status": "success",  
3   "message": "Task deleted successfully"  
4 }
```

Notification Service

- No public REST endpoints.
- Consumes Kafka events such as:
 - TaskAssigned
 - ReminderScheduled
 - IssueReported
 - EventDayAlertSends email/SMS notifications to service teams and event manager.

8. Security

Authentication

- JWT-based authentication for the event manager.

Authorization

Only one role:

- **Event Manager:** Full access to event and task management features.

Transport Layer Security

- All communication over HTTPS.

9. Key Design Decisions

- **Microservices Architecture:** Independent scalability of event management and notification workloads.
- **Kafka Messaging:** Guarantees reliable reminders even under network load.
- **Akka Actors:** Enables high-performance asynchronous reminders and real-time alerts.
- **Dockerization:** Simplifies deployment while keeping all environments consistent.

10. Conclusion

This architecture provides a scalable and reliable solution for coordinating multiple event service teams. The system ensures timely task assignment, continuous progress tracking, and automated reminders, allowing the event manager to maintain complete control over event preparation. With modular microservices and robust messaging, it is designed for future expansion and real-world efficiency.